Python Programming - Week 02

Control Statements (Continued from Week 01)

Conditional Statements

Conditional statements allow your program to make decisions based on certain conditions.

```
# Basic if statement
age = 18
if age >= 18:
    print("You are an adult")

# if-else statement
temperature = 15
if temperature > 25:
    print("It's warm outside")
else:
    print("It's cool outside")

# if-elif-else statement
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
print(f"Your grade is {grade}")
```

Loops

Loops allow you to execute a block of code multiple times.

For Loop

```python
# Looping through a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# Looping with range
for i in range(5):  # 0,1,2,3,4
    print(i)

# Looping through a dictionary
person = {"name": "Alice", "age": 30, "city": "New York"}
for key, value in person.items():
    print(f"{key}: {value}")
```

While Loop

```python
# Basic while loop
count = 0
while count < 5:
    print(count)
    count += 1

# Using break to exit a loop
while True:
    user_input = input("Enter 'quit' to exit: ")
    if user_input == "quit":
        break
    print(f"You entered: {user_input}")
```

Operators in Python

Python supports a variety of operators for performing operations on variables and values.

Arithmetic Operators

Used for mathematical operations:

Operator	Description	Example
`+`	Addition	`x + y`
`-`	Subtraction	`x - y`
`*`	Multiplication	`x * y`
`/`	Division (results in float)	`x / y`
`%`	Modulus (remainder)	`x % y`
`**`	Exponentiation	`x ** y`
`//`	Floor Division	`x // y`

a, b = 10, 3

```
print(f"Addition: {a + b}")         # 13
print(f"Subtraction: {a - b}")      # 7
print(f"Multiplication: {a * b}")   # 30
print(f"Division: {a / b}")         # 3.3333...
print(f"Modulus: {a % b}")          # 1
print(f"Exponentiation: {a ** b}")  # 1000
print(f"Floor Division: {a // b}")  # 3
```

Comparison Operators

Used to compare values:

Operator	Description	Example
`==`	Equal	`x == y`
`!=`	Not equal	`x != y`

`>`	Greater than	`x > y`
`<`	Less than	`x < y`
`>=`	Greater than or equal to	`x >= y`
`<=`	Less than or equal to	`x <= y`

x, y = 10, 5

```
print(f"Equal: {x == y}")              # False
print(f"Not Equal: {x != y}")          # True
print(f"Greater Than: {x > y}")        # True
print(f"Less Than: {x < y}")           # False
print(f"Greater or Equal: {x >= y}")   # True
print(f"Less or Equal: {x <= y}")      # False
```

Logical Operators

Used to combine conditional statements:

Operator	Description	Example
`and`	Returns True if both statements are true	`x and y`
`or`	Returns True if one of the statements is true	`x or y`
`not`	Reverses the result	`not x`

p, q = True, False

```
print(f"p and q: {p and q}")  # False
print(f"p or q: {p or q}")    # True
print(f"not p: {not p}")      # False
```

Assignment Operators

Used to assign values to variables:

Operator	Description	Example	Equivalent to
`=`	Simple assignment	`x = 5`	-
`+=`	Add and assign	`x += 3`	`x = x + 3`
`-=`	Subtract and assign	`x -= 3`	`x = x - 3`
`*=`	Multiply and assign	`x *= 3`	`x = x * 3`
`/=`	Divide and assign	`x /= 3`	`x = x / 3`
`%=`	Modulus and assign	`x %= 3`	`x = x % 3`
`**=`	Exponentiate and assign	`x **= 3`	`x = x ** 3`
`//=`	Floor divide and assign	`x //= 3`	`x = x // 3`

```
x = 10
x += 5  # Same as x = x + 5
print(x)  # 15

y = 20
y *= 2  # Same as y = y * 2
print(y)  # 40
```

Identity Operators

Used to compare objects:

Operator	Description	Example
`is`	Returns True if both variables are the same object	`x is y`
`is not`	Returns True if both variables are not the same object	`x is not y`

a = [1, 2, 3]

b = [1, 2, 3]
c = a

```
print(a is b)      # False (different objects with same content)
print(a is c)      # True (same object)
print(a is not b)  # True
```

Membership Operators

Used to test if a value is present in a sequence:

Operator	Description	Example
`in`	Returns True if a sequence with the specified value is present in the object	`x in y`
`not in`	Returns True if a sequence with the specified value is not present in the object	`x not in y`

```
fruits = ["apple", "banana", "cherry"]
print("apple" in fruits)       # True
print("orange" in fruits)      # False
print("orange" not in fruits)  # True
```

Bitwise Operators

Used to compare binary numbers:

Operator	Description	Example
`&`	AND	`x & y`
`|`	OR	`x | y`
`^`	XOR	`x ^ y`
`~`	NOT	`~x`
`<<`	Left Shift	`x << y`

>>	Right Shift	x >> y

```
a, b = 10, 4  # In binary: a = 1010, b = 0100
print(f"a & b: {a & b}")   # 0 (0000 in binary)
print(f"a | b: {a | b}")   # 14 (1110 in binary)
print(f"a ^ b: {a ^ b}")   # 14 (1110 in binary)
print(f"~a: {~a}")         # -11 (bitwise complement)
print(f"a << 1: {a << 1}") # 20 (left shift by 1 position)
print(f"a >> 1: {a >> 1}") # 5 (right shift by 1 position)
```

Functions in Python

Functions are blocks of reusable code designed to perform a specific task. They help in organizing code, improving readability, and avoiding repetition.

Defining and Calling Functions

```
# Defining a basic function
def greet():
    print("Hello, World!")

# Calling the function
greet()  # Output: Hello, World!

# Function with parameters
def greet_person(name):
    print(f"Hello, {name}!")

greet_person("Alice")  # Output: Hello, Alice!

# Function with return value
def add(a, b):
    return a + b

result = add(5, 3)
print(result)  # Output: 8
```

Function Parameters

Python functions can have various types of parameters:

Required Parameters

```
def multiply(x, y):
    return x * y

print(multiply(2, 3))  # Output: 6
```

Default Parameters

```
def power(base, exponent=2):
    return base ** exponent

print(power(5))      # Output: 25 (uses default exponent=2)
print(power(5, 3))   # Output: 125 (uses provided exponent=3)
```

Keyword Arguments

```
def describe_pet(animal_type, pet_name):
    print(f"I have a {animal_type} named {pet_name}.")

# Using keyword arguments
describe_pet(animal_type="dog", pet_name="Rex")
describe_pet(pet_name="Whiskers", animal_type="cat")  # Order doesn't matter
```

Variable-Length Arguments

```
# *args - variable number of positional arguments
def sum_all(*numbers):
    total = 0
    for num in numbers:
        total += num
    return total

print(sum_all(1, 2, 3, 4, 5))  # Output: 15

# **kwargs - variable number of keyword arguments
def build_profile(**user_info):
    profile = {}
```

```
    for key, value in user_info.items():
        profile[key] = value
    return profile

user = build_profile(name="Alice", age=30, occupation="Developer")
print(user)

# Output: {'name': 'Alice', 'age': 30, 'occupation': 'Developer'}
```

Lambda Functions

Lambda functions are small anonymous functions defined with the `lambda` keyword. They can take any number of arguments but can only have one expression.

```
# Regular function
def square(x):
    return x ** 2

# Equivalent lambda function
square_lambda = lambda x: x ** 2

print(square(5))        # Output: 25
print(square_lambda(5)) # Output: 25

# Lambda functions are often used with higher-order functions
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x**2, numbers))
print(squared)  # Output: [1, 4, 9, 16, 25]

# Using lambda with filter
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers)  # Output: [2, 4]
```

Scope of Variables

The scope of a variable determines where in your code you can access that variable.

```
# Global variable
global_var = "I am global"
```

```
def test_scope():
    # Local variable
    local_var = "I am local"
    print(global_var)  # Can access global variable
    print(local_var)   # Can access local variable

test_scope()
print(global_var)      # Can access global variable
# print(local_var)     # Error: local_var is not defined

# Modifying global variable inside a function
def modify_global():
    global global_var
    global_var = "Modified global"

modify_global()
print(global_var)  # Output: Modified global
```

Modules in Python

A module is a file containing Python code that can be imported and used in other Python programs. Modules help organize code into manageable pieces and promote code reuse.

Using Built-in Modules

Python comes with a large standard library of modules:

```
# Using the math module
import math

print(math.pi)          # Output: 3.141592653589793
print(math.sqrt(16))    # Output: 4.0
print(math.floor(4.7))  # Output: 4

# Importing specific functions from a module
from math import ceil, floor

print(ceil(4.2))   # Output: 5
print(floor(4.7))  # Output: 4

# Importing with alias
```

```python
import math as m

print(m.pi)  # Output: 3.141592653589793

# Importing all functions (not recommended in practice)
from math import *

print(pi)       # Output: 3.141592653589793
print(sqrt(16)) # Output: 4.0
```

Creating Your Own Modules

You can create your own modules by saving Python code in a file:

1. Create a file named `mymath.py`:

```python
# mymath.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

PI = 3.14159
```

2. Import and use your module in another file:

```python
# main.py
import mymath

result1 = mymath.add(10, 5)
result2 = mymath.subtract(10, 5)

print(result1)     # Output: 15
print(result2)     # Output: 5
print(mymath.PI)   # Output: 3.14159
```

Module Search Path

Python looks for modules in the following locations:

1. The directory containing the current script
2. The directories in the PYTHONPATH environment variable
3. The standard library directories
4. The site-packages directory

You can view the search path:

import sys
print(sys.path)

Packages in Python

A package is a way of organizing related modules into a directory hierarchy. Packages help in structuring large code bases.

Creating a Package

1. Create a directory structure:

```
math_operations/
    __init__.py
    basic/
        __init__.py
        arithmetic.py
    advanced/
        __init__.py
        calculus.py
```

2. Content of files:

```
# math_operations/__init__.py
# This file makes the directory a package
# Can be empty or contain initialization code

# math_operations/basic/__init__.py
# Makes basic directory a subpackage

# math_operations/basic/arithmetic.py
```

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

# math_operations/advanced/__init__.py
# Makes advanced directory a subpackage

# math_operations/advanced/calculus.py
def derivative(f, x, h=0.0001):
    return (f(x + h) - f(x)) / h
```

3. Using the package:

```
# Import a module from the package
import math_operations.basic.arithmetic

result = math_operations.basic.arithmetic.add(5, 3)
print(result)  # Output: 8

# Shorter import
from math_operations.basic.arithmetic import subtract

result = subtract(10, 4)
print(result)  # Output: 6

# Importing from subpackage
from math_operations.advanced import calculus
```

Define a function to find the derivative of

```
def f(x):
    return x**2

derivative_at_2 = calculus.derivative(f, 2)
print(derivative_at_2)  # Output: ~4.0001 (approximates derivative of x^2 at x=2)
```

Popular Python Packages

Python has thousands of third-party packages available. Here are a few popular ones:

- **NumPy**: For numerical computing
- **pandas**: For data analysis and manipulation
- **matplotlib**: For data visualization
- **requests**: For HTTP requests
- **Flask** and **Django**: For web development

These can be installed using pip, Python's package installer:

```
pip install numpy pandas matplotlib requests flask django
```

Environment Variables in Python

Environment variables are variables that exist outside of your Python program, in your operating system's environment. They can be used to store configuration settings, API keys, or other values that shouldn't be hardcoded.

Accessing Environment Variables

import os

```
# Get an environment variable
username = os.environ.get('USERNAME')  # Returns None if not set
print(f"Current user: {username}")

# Get with default value
database_url = os.environ.get('DATABASE_URL', 'sqlite:///default.db')
print(f"Database URL: {database_url}")

# Get using dictionary access
try:
    home_dir = os.environ['HOME']  # Raises KeyError if not set
    print(f"Home directory: {home_dir}")
except KeyError:
    print("HOME environment variable not set")
```

Setting Environment Variables

Environment variables can be set:

1. **Temporarily in Python**:

import os

```
# Set for current process only
os.environ['APP_MODE'] = 'development'
print(os.environ['APP_MODE'])  # Output: development
```

2. **From the command line** (before running the script):
- Windows: `set APP_MODE=development`
- macOS/Linux: `export APP_MODE=development`
3. **In a `.env` file** (requires additional library):

```
# .env file
APP_MODE=development
API_KEY=your_secret_key

# Using python-dotenv library
from dotenv import load_dotenv
load_dotenv()  # Load environment variables from .env

import os
mode = os.environ.get('APP_MODE')
api_key = os.environ.get('API_KEY')
```

Common Use Cases

- Storing API keys and secrets
- Setting different configurations for development/production environments
- Database connection strings
- File paths that differ between systems
- Toggle features on/off based on environment

Command Line Arguments in Python

Command line arguments allow users to pass information to a script when it's executed from the command line.

Using `sys.argv`

The most basic way to handle command line arguments is through the `sys.argv` list:

```
# script.py
import sys

# sys.argv[0] is the script name
# sys.argv[1:] are the arguments

print(f"Script name: {sys.argv[0]}")
print(f"Arguments: {sys.argv[1:]}")

if len(sys.argv) > 1:
    name = sys.argv[1]
    print(f"Hello, {name}!")
else:
    print("Please provide your name as an argument")

Running this script:
python script.py Alice

Output:
Script name: script.py
Arguments: ['Alice']
Hello, Alice!
```

Using the `argparse` Module

For more complex argument parsing, the `argparse` module provides a more structured approach:

```
# calculator.py
import argparse

parser = argparse.ArgumentParser(description="Simple calculator program")

# Add arguments
parser.add_argument("operation", choices=["add", "subtract", "multiply", "divide"],
                    help="Math operation to perform")
parser.add_argument("x", type=float, help="First number")
```

```
parser.add_argument("y", type=float, help="Second number")
parser.add_argument("--verbose", "-v", action="store_true", help="Provide verbose
output")

# Parse arguments
args = parser.parse_args()

# Perform calculation
result = None
if args.operation == "add":
    result = args.x + args.y
elif args.operation == "subtract":
    result = args.x - args.y
elif args.operation == "multiply":
    result = args.x * args.y
elif args.operation == "divide":
    if args.y == 0:
        print("Error: Division by zero")
        exit(1)
    result = args.x / args.y

# Output
if args.verbose:
    print(f"Performing {args.operation} on {args.x} and {args.y}")
print(f"Result: {result}")
```

Running this script:

```
python calculator.py add 5 3

Output:
Result: 8.0

With verbose flag:
python calculator.py multiply 5 3 --verbose

Output:
Performing multiply on 5.0 and 3.0
Result: 15.0
```

Getting help:

python calculator.py --help

Output:

usage: calculator.py [-h] [--verbose] {add,subtract,multiply,divide} x y

Simple calculator program

positional arguments:
  {add,subtract,multiply,divide}
                        Math operation to perform
  x   First number
  y   Second number

optional arguments:
  -h, --help   show this help message and exit
  --verbose, -v   Provide verbose output

Practical Examples

Example 1: Creating a Simple Calculator Module

```
# calculator.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b

if __name__ == "__main__":
    # This code runs only when the file is executed directly
```

```
    print("Calculator module")
    print(f"5 + 3 = {add(5, 3)}")
    print(f"5 - 3 = {subtract(5, 3)}")
    print(f"5 * 3 = {multiply(5, 3)}")
    print(f"5 / 3 = {divide(5, 3)}")
```

Example 2: A Command-Line To-Do List Application

```
# todo.py
import argparse
import json
import os

TODO_FILE = "todos.json"

def load_todos():
    if os.path.exists(TODO_FILE):
        with open(TODO_FILE, 'r') as f:
            return json.load(f)
    return []

def save_todos(todos):
    with open(TODO_FILE, 'w') as f:
        json.dump(todos, f, indent=2)

def list_todos(args):
    todos = load_todos()
    if not todos:
        print("No tasks found")
        return

    print("Tasks:")
    for idx, todo in enumerate(todos, 1):
        status = "✓" if todo.get("completed") else " "
        print(f"{idx}. [{status}] {todo['task']}")

def add_todo(args):
    todos = load_todos()
    todo = {"task": args.task, "completed": False}
    todos.append(todo)
    save_todos(todos)
```

```
    print(f"Added task: {args.task}")

def complete_todo(args):
    todos = load_todos()
    if 1 <= args.task_id <= len(todos):
        todos[args.task_id - 1]["completed"] = True
        save_todos(todos)
        print(f"Marked task {args.task_id} as completed")
    else:
        print("Invalid task ID")

def delete_todo(args):
    todos = load_todos()
    if 1 <= args.task_id <= len(todos):
        deleted = todos.pop(args.task_id - 1)
        save_todos(todos)
        print(f"Deleted task: {deleted['task']}")
    else:
        print("Invalid task ID")

def main():
    parser = argparse.ArgumentParser(description="Simple ToDo List
Manager")
    subparsers = parser.add_subparsers(dest="command", help="Command to
run")

    # List command
    list_parser = subparsers.add_parser("list", help="List all tasks")
    list_parser.set_defaults(func=list_todos)

    # Add command
    add_parser = subparsers.add_parser("add", help="Add a new task")
    add_parser.add_argument("task", help="Task description")
    add_parser.set_defaults(func=add_todo)

    # Complete command
    complete_parser = subparsers.add_parser("complete", help="Mark task as
completed")
    complete_parser.add_argument("task_id", type=int, help="Task ID to mark
as completed")
    complete_parser.set_defaults(func=complete_todo)

    # Delete command
```

```
    delete_parser = subparsers.add_parser("delete", help="Delete a task")
    delete_parser.add_argument("task_id", type=int, help="Task ID to
delete")
    delete_parser.set_defaults(func=delete_todo)

    args = parser.parse_args()

    if args.command is None:
        parser.print_help()
    else:
        args.func(args)

if __name__ == "__main__":
    main()
```

Usage examples:

```
# Add tasks
python todo.py add "Learn Python functions"
python todo.py add "Create a module"

# List tasks
python todo.py list

# Mark task as completed
python todo.py complete 1

# Delete task
python todo.py delete 2
```

Example 3: Using Environment Variables for Configuration

```
# config.py
import os
from dotenv import load_dotenv

# Load environment variables from .env file if present
load_dotenv()

# Application configuration
```

```
CONFIG = {
    "database": {
        "host": os.environ.get("DB_HOST", "localhost"),
        "port": int(os.environ.get("DB_PORT", "5432")),
        "name": os.environ.get("DB_NAME", "myapp"),
        "user": os.environ.get("DB_USER", "postgres"),
        "password": os.environ.get("DB_PASSWORD", ""),
    },
    "app": {
        "debug": os.environ.get("APP_DEBUG", "false").lower() == "true",
        "secret_key": os.environ.get("APP_SECRET_KEY",
"default-secret-key"),
        "log_level": os.environ.get("LOG_LEVEL", "INFO"),
    }
}

def get_config():
    return CONFIG

if __name__ == "__main__":
    # Print current configuration when run directly
    import json
    # Hide password when printing
    safe_config = dict(CONFIG)
    if "password" in safe_config["database"]:
        safe_config["database"]["password"] = "********" if
safe_config["database"]["password"] else ""

    print("Current configuration:")
    print(json.dumps(safe_config, indent=2))

Example .env file:
DB_HOST=production-db.example.com
DB_NAME=production
DB_USER=admin
APP_DEBUG=false
APP_SECRET_KEY=my-secure-secret-key
```

Summary

In Python Programming - Week 02, we've covered:

1. **Operators**:
    - Arithmetic (+, -, *, /, %, **, //)
    - Comparison (==, !=, >, <, >=, <=)
    - Logical (and, or, not)
    - Assignment (=, +=, -=, etc.)
    - Identity (is, is not)
    - Membership (in, not in)
    - Bitwise (&, |, ^, ~, <<, >>)
2. **Functions**:
    - Defining and calling functions
    - Parameters (required, default, keyword, variable-length)
    - Return values
    - Lambda functions
    - Variable scope
3. **Modules**:
    - Using built-in modules
    - Creating your own modules
    - Import statements
    - Module search path
4. **Packages**:
    - Package structure
    - Creating packages
    - Importing from packages
    - Popular Python packages
5. **Environment Variables**:
    - Accessing variables with os.environ
    - Setting variables
    - Using .env files
    - Common use cases
6. **Command Line Arguments**:
    - Basic argument handling with sys.argv
    - Advanced argument parsing with argparse
    - Creating command-line interfaces

7. **Practical Examples**:
    - Calculator module
    - Command-line to-do list application
    - Configuration using environment variables

These concepts build upon what you learned in Week 01 and provide you with the tools to create more complex and modular Python applications.